

PROGRAM-8

Create a kubernetes Pods using YAML Descriptors

Project Structure

Program-8

- [server.js](#)
- nodejs-pod.yaml
- nodejs-configmap.yaml

Step1: Start minikube

```
1RV24MC018_Ananya_h@user-ThinkPad-E480:~/labprograms/Program-7$ minikube start
minikube v1.37.0 on Ubuntu 22.04
Using the docker driver based on existing profile
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.48 ...
Restarting existing docker container for "minikube" ...

Docker is nearly out of disk space, which may cause deployments to fail! (92% of capacity). You can pass '--force' to skip this check.
Suggestion:

Try one or more of the following to free up space on the device:

1. Run "docker system prune" to remove unused Docker data (optionally with "--a")
2. Increase the storage allocated to Docker for Desktop by clicking on:
Docker icon > Preferences > Resources > Disk Image Size
3. Run "minikube ssh -- docker system prune" if using the Docker container runtime
Related issue: https://github.com/kubernetes/minikube/issues/9024

Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
Verifying Kubernetes components...
  ■ Using image registry.k8s.io/metrics-server/metrics-server:v0.8.0
  ■ Using image gcr.io/k8s-minikube/storage-provisioner:v5
Enabled addons: metrics-server, storage-provisioner, default-storageclass

! /usr/bin/kubectl is version 1.28.15, which may have incompatibilities with Kubernetes 1.34.0.
  ■ Want kubectl v1.34.0? Try 'minikube kubectl -- get pods -A'
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Step2: Check the status of the minikube

```
1RV24MC018_Ananya_h@user-ThinkPad-E480:~/labprograms/Program-7$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Step3: Create a [server.js](#) file

```
GNU nano 6.2 server.js
const http=require("http");
const port=3000
const server=http.createServer((res,req)=>{
res.end("hello from node js running in kuberentes cluster");
});
server.listen(port,()=>{
console.log(`Server running in http://localhost:${port}`);
});
```

Step 4: Create a nodejs-configmap.yaml file

```
GNU nano 6.2 nodejs-configmap.y
apiVersion: v1
kind: ConfigMap
metadata:
  name: nodejs-app-config
data:
  server.js: |
    const http = require("http");
    const port = 3000;
    const server = http.createServer((req, res) => {
      res.end("hello from the node js running inside the kubernetes cluster");
    });
    server.listen(port, () => {
      console.log(`server running on port ${port}`);
    });
```

Step 5: Create a nodejs-pod.yaml file

```
GNU nano 6.2
apiVersion: v1
kind: Pod
metadata:
  name: program-8
  labels:
    app: nodejs
spec:
  containers:
    - name: nodejs
      image: node:18
      command: ["node", "/usr/src/app/server.js"]
      volumeMounts:
        - name: app-code
          mountPath: /usr/src/app
      ports:
        - containerPort: 3000
  volumes:
    - name: app-code
      configMap:
        name: nodejs-app-config
```

Step 6: Apply the yaml files

```
1RV24MC018_Ananya_h@user-ThinkPad-E480:~/labprograms/Program-8$ kubectl apply -f nodejs-pod.yaml
pod/program-8 created
1RV24MC018_Ananya_h@user-ThinkPad-E480:~/labprograms/Program-8$ kubectl apply -f nodejs-configmap.yaml
configmap/nodejs-app-config unchanged
```

Step 7: Check the status

```
1RV24MC018_Ananya_h@user-ThinkPad-E480:~/labprograms/Program-8$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-6f9664446b-k2cmb   1/1     Running   0           11m
nginx-deployment-6f9664446b-qc9wl   1/1     Running   0           11m
program-8                            1/1     Running   0           95s
```

Step 8: Access the web Application

```
1RV24MC018_Ananya_h@user-ThinkPad-E480:~/labprograms/Program-8$ kubectl port-forward pod/program-8 3000:3000
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
Handling connection for 3000
Handling connection for 3000
```

